기본 RAG 체인 구성과 검증 (실습)

4-1. Retriever 만들기 (실습)
4-2. RAG 체인 조립 (실습)
4-3. 동작 검증 (실습)

4-1. Retriever 만들기 (실습)

> 벡터 저장소에서 검색기를 만들고 k값에 따라 결과가 어떻게 달라지는지 관찰해봅니다.

`similarity_search` 는 그냥 함수라서 LCEL 체인에 끼울 수 없습니다. `as_retriever()` 로 감싸면 Runnable 객체가 되어 1일차에서 배운 파이프로 이을 수 있습니다. 가장 중요한 조정값은 k(가져올 청크 수)입니다.

- k가 작으면(1-2) 노이즈는 적지만 정답 청크를 놓칠 위험이 커집니다.
- k가 크면(8-10) 정답이 포함될 확률은 높지만 무관한 청크가 섞여 LLM이 헷갈리고 토큰 비용이 늘어납니다.
- 출발점은 k=3-5이고 이후 6번째 세션에서 만들 평가 세트로 측정하며 조정합니다.

따라하기

1. 아래 코드를 `retriever.py` 로 저장하고 k값을 바꿔가며 결과를 비교합니다.

```
# retriever.py - Retriever 생성과 k값 조정
from dotenv import load_dotenv
from langchain_aws import BedrockEmbeddings
from langchain_chroma import Chroma

load_dotenv()
embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0",
    region_name="us-east-1",
)
db = Chroma(
    collection_name="sds_policies",
    embedding_function=embeddings,
    persist_directory="./chroma_db",
)

retriever = db.as_retriever(search_kwargs={"k": 3})
docs = retriever.invoke("연차 신청은 어떻게 하나요?")
for d in docs:
    print(f"[{d.metadata['source']}] {d.page_content[:50]}...")

# k값에 따른 차이 관찰
for k in [1, 3, 5]:
    r = db.as_retriever(search_kwargs={"k": k})
    hits = r.invoke("연차 이월 규정")
    print(f"k={k}: {[h.metadata['source'] for h in hits]}")

# MMR - 넓은 질문에서 여러 문서를 고르게
mmr = db.as_retriever(search_type="mmr", search_kwargs={"k": 3, "fetch_k":
6})
print("MMR:", [h.metadata["source"] for h in mmr.invoke("회사 규정 전반")])
```

2. 출력을 관찰합니다. k가 커지면 어느 지점부터 무관한 문서가 끌려 들어오기 시작합니다. MMR(Maximal Marginal Relevance, 최대 한계 관련성)은 후보 fetch_k개 중에서 서로 다양한 k개를 고르므로 "회사 규정 전반" 같은 넓은 질문에서 여러 문서가 고르게 나옵니다.

막힐 때

- 검색 결과가 0개입니다.

  3번째 세션 3-3의 적재를 건너뛰었거나 collection_name이 다른 경우입니다. `load_db.py` 로 청크 수부터 확인합니다.

확인: k=1, 3, 5에서 검색되는 파일 구성이 달라지는 것을 보면 성공입니다.

k값이 정답 포함 확률과 노이즈 사이의 조절 손잡이라는 걸 확인했습니다.

4-2. RAG 체인 조립 (실습)

검색 결과를 프롬프트에 주입해 출처가 표시되는 RAG 체인을 완성해봅니다.

이제 부품이 다 모였습니다. 질문이 들어온 뒤 답이 나오기까지 순서는 다음과 같습니다.

1. `retriever`가 질문과 관련된 청크를 찾습니다.
2. `format_docs`가 청크 리스트를 출처 표시가 붙은 문자열로 합칩니다.
3. 프롬프트가 그 문자열을 "이 문서만 참고해서 답하라"는 지시와 함께 묶습니다.
4. LLM이 답을 생성하고 파서가 문자열로 꺼냅니다.

프롬프트의 "문서에 없는 내용은 모른다고 답하라"가 환각을 막는 장치입니다.

따라하기

1. 아래 코드를 `rag_chain.py` 로 저장하고 실행합니다. 파일명을 바꾸면 안 됩니다. 이후의 검증, 검색 실패 재현, 하이브리드 검색, 평가 실습이 전부 이 파일을 import합니다.

```
# rag_chain.py - 기본 RAG 체인
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse, BedrockEmbeddings
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

load_dotenv()

llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
    temperature=0,
)
embeddings = BedrockEmbeddings(
    model_id="amazon.titan-embed-text-v2:0",
    region_name="us-east-1",
)

db = Chroma(
    collection_name="sds_policies",
    embedding_function=embeddings,
    persist_directory="./chroma_db",
)
retriever = db.as_retriever(search_kwargs={"k": 3})

prompt = ChatPromptTemplate.from_messages([
    ("system",
     "너는 삼성SDS 사내 정책 안내 어시스턴트야. 아래 사내 문서만 참고해 답해. "
     "문서에 없는 내용은 반드시 '사내 문서에서 찾을 수 없습니다'라고 답해. "
     "답변 끝에 참고한 문서를 '(출처: 파일명)' 형식으로 붙여.\n\n"
     "--- 사내 문서 ---\n{context}"),
    ("human", "{question}"),
])


def format_docs(docs):
    return "\n\n---\n\n".join(
        f"[{d.metadata.get('source', '?')}] {d.page_content}" for d in docs
    )


rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
```

```
    | StrOutputParser()
)

if __name__ == "__main__":
    print(rag_chain.invoke("클라우드운영팀 김하늘입니다. 연차 이월은 며칠까지 되나요?"))
```

2. 답변에 이월 5일이라는 정확한 수치와 출처가 함께 나오는지 확인합니다.

체인 첫 줄의 dict는 입력이 두 갈래로 갈라진다고 읽으면 됩니다. `context` 키로는 검색과 포맷을 거친 결과가 들어가고 `question` 키로는 입력이 그대로(`RunnablePassthrough`) 들어갑니다. 그 dict가 프롬프트의 `{context}` 와 `{question}` 자리를 채웁니다.

막힐 때

- `KeyError: 'context'`

  프롬프트의 변수 이름과 dict의 키 이름이 정확히 일치해야 합니다. 오타 한 글자면 이 오류가 납니다.

- LLM이 지시를 무시하고 아는 척을 합니다.

  "모른다고 답하라"는 지시를 human 메시지에 넣으면 잘 무시됩니다. 반드시 system에 넣고 "반드시" 같은 강조 표현을 씁니다.

확인: 답변에 문서의 실제 수치가 나오고 끝에 출처가 붙으면 성공입니다.

> 검색 결과를 프롬프트에 주입하는 것만으로 LLM이 사내 규정을 정확히 답하게 된다는 걸 확인했습니다.

4-3. 동작 검증 (실습)

> 같은 질문을 RAG 유무로 나란히 비교하고 근거 문서를 함께 반환하게 만들어봅니다.

같은 질문을 RAG 없는 LLM과 RAG 체인에 각각 던져 나란히 비교합니다.

따라하기

1. 아래 코드를 `verify.py` 로 저장하고 문서에 답이 있는 질문 두 개와 없는 질문 하나로 비교합니다.

```
# verify.py - RAG 유무 비교 검증
from rag_chain import rag_chain, llm


def get_text(message):
    """ChatBedrockConverse는 content를 블록 리스트로 주기도 하므로 텍스트만 모아 반환합니
다."""
    content = message.content
    if isinstance(content, list):
        return "".join(block.get("text", "") for block in content if isinsta
nce(block, dict))
    return content


questions = [
    "국내 출장 식비 한도는 하루 얼마인가요?",   # 문서에 답 있음
    "재택근무는 주 며칠까지 가능한가요?",       # 문서에 답 있음
    "육아휴직은 얼마나 쓸 수 있나요?",          # 문서에 답 없음 -> 모른다고 해야 정답
]

for q in questions:
    print("=" * 60)
    print(f"질문: {q}")
    print(f"\n[RAG 없음] {get_text(llm.invoke(q))[:100]}")
    print(f"\n[RAG 적용] {rag_chain.invoke(q)}")
```

2. 세 가지를 확인합니다.
   - 문서에 있는 질문에 RAG가 정확한 수치를 답하는가.
   - 문서에 없는 질문을 거절하는가. 일반론으로 때우면 실패입니다.
   - 답변 끝에 출처가 붙는가.

3. 출처를 프롬프트에만 맡기지 않고 코드로 강제하고 싶다면 검색 결과를 함께 반환하도록 체인을 바꿉니다. 아래 코드는 `verify_sources.py`로 저장하고 실행합니다.

```
# verify_sources.py - 근거 문서를 함께 반환
from langchain_core.runnables import RunnableParallel
from rag_chain import rag_chain, retriever

rag_with_sources = RunnableParallel(
    answer=rag_chain,
    sources=retriever | (lambda docs: sorted({d.metadata["source"] for d in
docs})),
)
result = rag_with_sources.invoke("물류플랫폼팀 박도윤입니다. 해외 출장 숙박비 한도가 얼마
인가요?")
print(result["answer"])
print("근거 문서:", result["sources"])
```

이 `sources` 필드는 6번째 세션 도전 미션의 자동 채점 기준으로 그대로 씁니다.

지금 만든 체인은 검색에서 생성으로 한 방향으로만 흐릅니다.

그래서 검색 결과가 엉뚱해도 그대로 답을 만들고, 질문이 모호해도 다시 묻지 않습니다.

이 한계는 3일차 LangGraph의 분기와 반복으로 풀고, 오늘은 5번째 세션에서 검색 품질을 개선합니다.

막힐 때

- no_answer 질문에 일반론으로 답합니다.

  system 프롬프트의 거절 지시가 약한 경우입니다. "반드시 '사내 문서에서 찾을 수 없습니다'라고만 답해"처럼 답변 문구 자체를 고정합니다.

- import 오류가 납니다.

  `verify.py`는 `rag_chain.py`와 같은 폴더에 있어야 합니다.

확인: RAG 적용 답변만 정확한 수치와 출처를 내고 문서에 없는 질문은 찾을 수 없다고 거절하면 성공입니다.

> RAG의 가치는 아는 것을 정확히 답하는 것과 모르는 것을 거절하는 것 두 가지라는 걸 확인했습니다.